

Yesterday 3:16 PM

Process vs thread.

Concurrency vs parallelism.

What is thread safety?

What is race condition?

What is a critical section?

What is synchronization?

synchronized method vs synchronized block.

What object is used as the monitor?

What happens when two threads access synchronized method?

What happens when two different objects call the same

Show more ▾



Absolutely. For a 5-year Java interview, don't just give definitions—explain **what happens internally** + a small Java example + **production relevance**.

1. Process vs Thread

Process

A **process** is an independent running program with its own memory space.

Example:

- Running IntelliJ → one process
- Running Chrome → another process

Each process has its own:

- Heap
- Stack
- Code/data
- OS resources

Thread

A **thread** is a lightweight execution unit inside a process.

Multiple threads within the same process **share the process heap**, but each thread has its own stack.

Process

- Heap (shared)
- Method Area
- Thread-1 → Stack
- Thread-2 → Stack
- Thread-3 → Stack



Interview answer

A process is an independent execution environment with its own memory space, while a thread is a lightweight execution unit within a process. Threads of the same process share heap memory but have their own stacks.

2. Concurrency vs Parallelism

Concurrency

Multiple tasks **make progress during overlapping periods**, but they don't necessarily execute at exactly the same time.

Example with one CPU core:

Time →

Task A: ■■■ ■■■

Task B: ■■■ ■■■



The CPU switches between them.

Parallelism

Multiple tasks actually execute **at the same time**, generally using multiple CPU cores.

Core 1: Task A ■■■■■■■■

Core 2: Task B ■■■■■■■■



Simple difference

Concurrency**Parallelism**

Dealing with multiple tasks

Executing multiple tasks simultaneously

Can happen on 1 core

Requires multiple execution resources

Context switching possible

Actual simultaneous execution

Interview answer

Concurrency means multiple tasks are in progress during overlapping time periods, whereas parallelism means multiple tasks are actually executing simultaneously, typically on different CPU cores.

3. What is Thread Safety?

A piece of code is **thread-safe** when multiple threads can execute it concurrently without causing incorrect or inconsistent results.

Example:

</> Java



```
class Counter {  
  
    private int count;  
  
    public synchronized void increment() {  
        count++;  
    }  
}
```

The `synchronized` keyword ensures that only one thread at a time executes `increment()` for the same object.

Without synchronization:

</> Java



```
count++;
```

is not actually one atomic operation.

Conceptually:

```
read count  
add 1  
write count
```



Two threads can interfere with each other.

Interview answer

Thread safety means shared data remains correct and consistent when accessed by multiple threads concurrently. We can achieve it using synchronization, locks, atomic classes, concurrent collections, immutability, or other concurrency mechanisms.

4. What is a Race Condition?

A **race condition** occurs when the result depends on the timing/order in which multiple threads execute.

Example:

</> Java



```
class Counter {  
  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Suppose:

Initial count = 0



```
Thread A → reads 0  
Thread B → reads 0  
Thread A → writes 1  
Thread B → writes 1
```

Expected:

2



Actual:

1



This is a race condition.

Interview answer

A race condition occurs when multiple threads access shared mutable state concurrently and the final result depends on their execution order.

5. What is a Critical Section?

A **critical section** is the part of code that accesses shared mutable data and therefore must not be executed concurrently by conflicting threads.

Example:

`</>` Java



```
void increment() {  
  
    synchronized (this) {  
        count++;  
    }  
}
```

Here:

`</>` Java



```
count++;
```

is the critical section.

Important

Not every piece of multithreaded code needs synchronization.

You generally need protection when threads access **shared mutable state**.

6. What is Synchronization?

Synchronization is a mechanism used to control access to shared resources so that multiple threads don't execute conflicting operations at the same time.

Java provides:

</> Java



synchronized

and explicit locking mechanisms such as:

</> Java



Lock

ReentrantLock

Example:

</> Java



```
public synchronized void increment() {  
    count++;  
}
```

Synchronization provides two important guarantees:

1. Mutual exclusion

Only one thread can execute the protected section at a time for the same monitor.

2. Visibility

Changes made before releasing the monitor become visible to a thread that subsequently acquires the same monitor.

7. synchronized Method vs synchronized Block

synchronized method

```
</> Java   
  
public synchronized void increment() {  
    count++;  
}
```

For an instance method, the lock is on:

```
</> Java   
  
this
```

The entire method is protected.

synchronized block

```
</> Java   
  
public void increment() {  
  
    // non-critical work  
  
    synchronized (this) {  
        count++;  
    }  
  
    // more work  
}
```

Only the required section is protected.

Why prefer synchronized block?

It gives you **finer-grained locking**.

For example:

```
</> Java   
  
public void process() {  
  
    doSomething(); // Doesn't need Lock
```

```
synchronized (this) {  
    updateSharedState();  
}  
  
doSomethingElse(); // Doesn't need Lock  
}
```

Instead of locking the entire method.

Interview answer

A synchronized method locks the entire method, whereas a synchronized block allows us to lock only the critical section. Synchronized blocks generally provide better control and can reduce lock contention.

8. What Object is Used as the Monitor?

This is an important interview question.

Every Java object can act as a **monitor**.

For:

```
</> Java  
  
public synchronized void test() {  
}  

```



the monitor is:

```
</> Java  
  
this
```



So:

```
</> Java  
  
synchronized void test()
```



is conceptually similar to:

</> Java



```
void test() {  
    synchronized (this) {  
        // method body  
    }  
}
```

For:

</> Java



```
synchronized (MyClass.class) {  
}
```

the monitor is the class object:

</> Java



```
MyClass.class
```

Example

</> Java



```
class Counter {  
  
    public synchronized void increment() {  
        // locks this object's monitor  
    }  
}
```

If:

</> Java



```
Counter c = new Counter();
```

then the monitor is associated with:

```
c
```



9. What happens when two threads access a synchronized method?

Suppose:

```
</> Java   
  
class Counter {  
  
    public synchronized void increment() {  
        // critical section  
    }  
  
}
```

And:

```
</> Java   
  
Counter counter = new Counter();  
  
Thread A → counter.increment()  
Thread B → counter.increment()
```

Both threads need the **same object's monitor**.

Suppose Thread A gets the lock first:

```
Thread A   
  ↓  
acquires counter's monitor  
  ↓  
executes increment()  
  ↓  
releases monitor
```

Thread B:

```
  
tries to acquire monitor  
  ↓  
waits  
  ↓  
gets monitor after A releases it  
  ↓  
executes method
```

So the method isn't executed concurrently by those two threads for that object.

10. What happens when two different objects call the same synchronized method?

This is a **very common** interview trap.

</> Java



```
Counter c1 = new Counter();  
Counter c2 = new Counter();
```

Suppose:

</> Java



```
public synchronized void increment() {  
    count++;  
}
```

Now:

Thread A → c1.increment()
Thread B → c2.increment()



They use **different monitors**:

c1 → Monitor 1
c2 → Monitor 2



Therefore, they **do not block each other**.

Both methods can execute simultaneously.

Thread A → locks c1 → executes
Thread B → locks c2 → executes



Key point

synchronized instance method locks **the current object**, not the method itself.

11. Instance Lock vs Class Lock

Instance Lock

Used by:

</> Java



```
public synchronized void method() {  
}
```

Lock:

</> Java



this

Example:

</> Java



```
Counter c1 = new Counter();  
Counter c2 = new Counter();
```

There are separate instance locks:

c1 → Lock A

c2 → Lock B



Therefore:

Thread 1 → c1.method()

Thread 2 → c2.method()



can run concurrently.

Class Lock

Used by:

</> Java



```
public static synchronized void method() {
}
```

The lock is associated with:

</> Java



Counter.class

Equivalent idea:

</> Java



```
synchronized (Counter.class) {
}
```

All instances share this class-level monitor.

Counter.class



```

|
├─ c1
├─ c2
└─ c3
```

If one thread holds the class lock, another thread trying to enter another `static synchronized` method on the same class must wait.

Important interview example

</> Java



```
class Test {

    public synchronized void instanceMethod() {
    }

    public static synchronized void staticMethod() {
    }

}
```

These use **different locks**:

instanceMethod()

↓



```
this
    staticMethod()
      ↓
Test.class
```

Therefore, one does not automatically block the other.

12. What is Deadlock?

A **deadlock** occurs when two or more threads wait indefinitely for locks held by each other.

Classic example:

</> Java

```
Object lock1 = new Object();
Object lock2 = new Object();
```

Thread A:

```
Lock lock1
  ↓
wait for lock2
```

Thread B:

```
Lock lock2
  ↓
wait for lock1
```

Result:

```
Thread A → holds lock1 → waiting for lock2
Thread B → holds lock2 → waiting for lock1
```

Neither can continue.

```
A ———waits————→ B's lock
↑                      ↓
└———waits————┘
```

13. How Can You Prevent Deadlock?

1. Maintain consistent lock ordering

Always acquire locks in the same order.

Bad:

```
Thread A: lock1 → lock2  
Thread B: lock2 → lock1
```



Good:

```
Thread A: lock1 → lock2  
Thread B: lock1 → lock2
```



This is one of the most important solutions.

2. Avoid unnecessary nested locks

Instead of:

```
</> Java  
  
synchronized (lock1) {  
    synchronized (lock2) {  
        // ...  
    }  
}
```



try to reduce the locking scope.

3. Use timed lock acquisition

With `ReentrantLock` :

```
</> Java  
  
if (lock.tryLock(1, TimeUnit.SECONDS)) {  
    try {
```



```
        // work
    } finally {
        lock.unlock();
    }
}
```

The thread doesn't have to wait forever.

4. Keep critical sections small

Don't hold a lock while performing slow operations such as unnecessary I/O or network calls.

Interview answer

Deadlocks can be prevented by maintaining consistent lock ordering, minimizing nested locks, keeping critical sections small, avoiding unnecessary locking, and using timed lock acquisition such as `tryLock()` where appropriate.

14. What is Livelock?

In a **livelock**, threads are not blocked—they are actively running, but they keep responding to each other and make **no useful progress**.

Think of two people trying to pass each other in a hallway:

Person A → moves right

Person B → moves right

A → moves left

B → moves left

A → right

B → right



Both are active, but neither progresses.

Deadlock vs Livelock

Deadlock	Livelock
Threads are blocked/waiting	Threads are actively executing
No progress	No useful progress
Waiting for locks/resources	Continuously responding/retrying
Usually easier to detect from thread dumps	Can be harder to recognize

15. What is Starvation?

Starvation occurs when a thread waits for a very long time because other threads continuously get access to the required resource.

Example:

```
Thread A → waiting for lock
Thread B → gets lock
Thread C → gets lock
Thread D → gets lock
Thread B → gets lock again
Thread C → gets lock again
...
Thread A → keeps waiting
```



Thread A isn't necessarily permanently deadlocked. Other threads are making progress, but A is being denied a fair opportunity.

Causes

- Unfair lock acquisition
- High-priority threads dominating resources
- Long-running synchronized sections
- Poor thread scheduling/design

★ Deadlock vs Livelock vs Starvation

This is worth memorizing for interviews:

Concept	What happens?	Are threads active?	Progress?
Deadlock	Threads wait for each other's locks	✗	✗
Livelock	Threads continuously react/retry	✓	✗
Starvation	One thread rarely gets resource access	Maybe	Others progress

Easy way to remember

Deadlock → "I'm waiting for you."
 Livelock → "I'm moving, but getting nowhere."
 Starvation → "Everyone gets a chance except me."



🔥 5-Year Experience Interview Summary

If the interviewer asks you to explain Java synchronization, a strong answer would be:

"In Java, synchronization is used to protect shared mutable state from concurrent access. An instance synchronized method acquires the monitor of the current object, while a static synchronized method acquires the monitor associated with the Class object. A synchronized block allows us to control exactly which object is used as the monitor and limits the locking scope. If two threads synchronize on the same monitor, one thread must wait until the other releases it. Different objects have different instance monitors, so their synchronized instance methods can execute concurrently. We need to be careful about race conditions, deadlocks, starvation, and livelocks when designing concurrent code."

That is the level of explanation I'd expect from a **5-year Java developer**.



What is thread contention?

What is volatile?

Does volatile make operations atomic?

volatile vs synchronized.

What is happens-before?

What is atomicity?

What is visibility?

What is ordering?

Explain race condition with an example.

How would you make a counter thread-safe?

For a 5-year Java interview, these are very important because interviewers often connect
volatile → visibility → happens-before → atomicity → race condition →
synchronization.

1. What is Thread Contention?

Thread contention occurs when multiple threads compete for the same shared resource, especially a lock.

Example:

</> Java

```
public synchronized void process() {
    // critical section
}
```



If 20 threads call `process()` on the **same object**, only one can hold the monitor at a time.

20 Threads

```
|
v
Lock
|
+----> Thread 1 gets lock
|
+----> Thread 2 waits
+----> Thread 3 waits
+----> Thread 4 waits
...
```



This is lock contention.

Why is contention bad?

Too much contention can cause:

- Threads waiting
- Context switching
- Lower throughput
- Increased latency
- CPU overhead

Interview answer

Thread contention happens when multiple threads compete for the same shared resource, such as a lock. Excessive contention can reduce application throughput and increase latency.

2. What is volatile ?

`volatile` is a Java keyword used mainly to guarantee **visibility of changes to a variable across threads** and to provide ordering guarantees around volatile accesses.

Example:

</> Java



```
class Task {  
  
    private volatile boolean running = true;  
  
    void stop() {  
        running = false;  
    }  
  
    void execute() {  
        while (running) {  
            // do work  
        }  
    }  
}
```

Without `volatile`, another thread may not reliably observe the updated value under the Java Memory Model.

With:

```
</> Java
```



```
volatile boolean running;
```

a write to `running` becomes visible to subsequent reads of that volatile variable.

Important

`volatile` does **not** make arbitrary compound operations atomic.

3. Does `volatile` Make Operations Atomic?

No.

This is one of the most common interview questions.

Consider:

```
</> Java
```



```
volatile int count = 0;
```

```
count++;
```

Many candidates say:

"Because `count` is volatile, `count++` is thread-safe."

✗ Wrong.

`count++` is effectively:

```
read count
```

```
↓
```

```
add 1
```

```
↓
```

```
write count
```



Two threads can interleave:

Initial count = 0



Thread A → read 0

Thread B → read 0

Thread A → write 1

Thread B → write 1

Final result:

1



instead of:

2



What does volatile provide?

volatile



|

+---- Visibility ✓

|

+---- Ordering guarantees ✓

|

+---- Atomicity of compound operations ✗

For a counter, use `AtomicInteger` or synchronization depending on the requirement.

4. volatile vs synchronized

volatile	synchronized
Mainly visibility + ordering	Mutual exclusion + visibility + ordering
Doesn't provide general mutual exclusion	Provides mutual exclusion
Doesn't make <code>count++</code> atomic	Can make <code>count++</code> atomic
No lock acquisition for the variable itself	Uses a monitor
Useful for simple shared state flags	Useful for critical sections

volatile**synchronized**

Usually lower coordination overhead

Can cause contention

Example where volatile is appropriate

</> Java 

```
private volatile boolean running;
```

One thread changes it:

</> Java 

```
running = false;
```

Other thread reads it:

</> Java 

```
while (running) {  
}
```

Example where synchronized is appropriate

</> Java 

```
private int count;  
  
public synchronized void increment() {  
    count++;  
}
```

Here we need **mutual exclusion** because `count++` is a compound operation.

5. What is Happens-Before?

Happens-before is a rule in the Java Memory Model that establishes an ordering and visibility relationship between actions.

If:

A happens-before B



then the effects of A are guaranteed to be visible to B, and A is ordered before B according to the memory model.

Common happens-before relationships

1. Program order

Within a single thread:

</> Java



```
int x = 10;  
int y = x;
```

The assignment to `x` happens-before the assignment/read involving `y` according to program order.

2. Unlock happens-before subsequent lock

If Thread A releases a monitor:

</> Java



```
synchronized (lock) {  
    x = 10;  
}
```

and Thread B subsequently acquires the **same monitor**:

</> Java



```
synchronized (lock) {  
    System.out.println(x);  
}
```

the unlock happens-before the subsequent lock.

3. Volatile write happens-before subsequent volatile read

</> Java



```
volatile boolean ready;
```

Thread A:

</> Java



```
ready = true;
```

Thread B:

</> Java



```
if (ready) {  
    // ...  
}
```

The volatile write happens-before a subsequent read of that same volatile variable.

4. Thread start

Actions before:

</> Java



```
thread.start();
```

happen-before actions performed by the started thread.

5. Thread termination / join

Actions performed by a thread happen-before another thread successfully returns from:

</> Java



```
thread.join();
```

Interview answer

Happens-before is a Java Memory Model guarantee that establishes both ordering and visibility between two actions. Common examples include program order, monitor unlock followed by a subsequent lock, volatile write followed by a volatile read, and actions before `Thread.start()` being visible to the started thread.

6. What is Atomicity?

An operation is **atomic** when it appears to happen as one indivisible operation.

For example:

</> Java



```
AtomicInteger counter = new AtomicInteger();  
  
counter.incrementAndGet();
```

The increment operation is atomic.

Another example:

</> Java



```
synchronized void increment() {  
    count++;  
}
```

The entire critical operation is protected from concurrent access by other threads using the same monitor.

Important distinction

</> Java



```
count++;
```

is **not atomic** for a normal integer shared between threads.

Conceptually:

READ → MODIFY → WRITE



Those steps can interleave.

7. What is Visibility?

Visibility means that when one thread changes shared data, another thread can reliably observe that change.

Example:

</> Java

```
private volatile boolean shutdown = false;
```



Thread A:

</> Java

```
shutdown = true;
```



Thread B:

</> Java

```
while (!shutdown) {  
    // work  
}
```



`volatile` ensures the updated value is visible according to the Java Memory Model.

Without proper synchronization

You cannot safely assume that another thread will immediately observe updates to ordinary shared variables.

Interview answer

Visibility means that changes made by one thread become observable by other threads according to the Java Memory Model. `volatile`, synchronization, locks, and certain concurrent utilities establish the required visibility guarantees.

8. What is Ordering?

Ordering means controlling the order in which memory operations are observed by threads.

Modern CPUs and compilers can reorder operations as long as single-threaded behavior remains correct.

For example:

</> Java

```
x = 10;  
ready = true;
```



Another thread might observe memory operations differently unless the program establishes the required synchronization relationship.

Using `volatile` can establish ordering guarantees around volatile accesses.

</> Java

```
x = 10;  
ready = true; // volatile
```



Then another thread:

</> Java

```
if (ready) { // volatile read  
    System.out.println(x);  
}
```



The synchronization relationship established through the volatile variable ensures the relevant preceding write is visible.

Simple way to remember

Atomicity → "Does it happen as one unit?"

Visibility → "Can another thread see my change?"

Ordering → "In what order can operations be observed?"



Happens-before → "What guarantees that relationship?"

9. Explain Race Condition with an Example

Consider:

</> Java



```
class Counter {  
  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

Suppose two threads execute:

</> Java



```
counter.increment();
```

You might expect:

Initial = 0



Thread A increments → 1

Thread B increments → 2

Final = 2

But `count++` isn't one operation.

It is conceptually:

```
read count  
add 1
```



```
write count
```

Possible execution:

```
Initial count = 0
```

Thread A	Thread B
read 0	
	read 0
write 1	
	write 1

Final:

```
1
```

Expected:

```
2
```

That's a race condition.

Fix using synchronized

```
</> Java

class Counter {

    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

Now only one thread at a time can execute the synchronized methods on the same Counter instance.

10. How Would You Make a Counter Thread-Safe?

There are several good approaches, and a 5-year candidate should know when to use each.

Approach 1: synchronized

</> Java



```
class Counter {  
  
    private int count;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int getCount() {  
        return count;  
    }  
}
```

Use when:

- You need multiple operations protected together.
- The critical section contains more than a simple atomic update.
- You want straightforward locking semantics.

Approach 2: AtomicInteger

For a simple counter, this is often a great choice.

</> Java



```
class Counter {  
  
    private final AtomicInteger count = new AtomicInteger();  
  
    public void increment() {  
        count.incrementAndGet();  
    }  
}
```

```
public int getCount() {  
    return count.get();  
}  
}
```

`AtomicInteger` uses atomic operations based on **CAS (Compare-And-Set)** and related JVM mechanisms rather than traditional monitor locking for these operations.

Good for:

Simple counters
Sequence numbers
Statistics
Concurrent state updates



Approach 3: ReentrantLock

</> Java



```
class Counter {  
  
    private int count;  
  
    private final Lock lock = new ReentrantLock();  
  
    public void increment() {  
        lock.lock();  
        try {  
            count++;  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```

Why use it?

`ReentrantLock` provides features that `synchronized` doesn't directly provide, such as:

- `tryLock()`
- timed lock acquisition
- interruptible lock acquisition
- optional fairness configuration

But don't automatically use it just because it's more advanced.

For a simple counter:

</> Java



`AtomicInteger`

is usually simpler.

★ Very Important Interview Comparison

If interviewer asks:

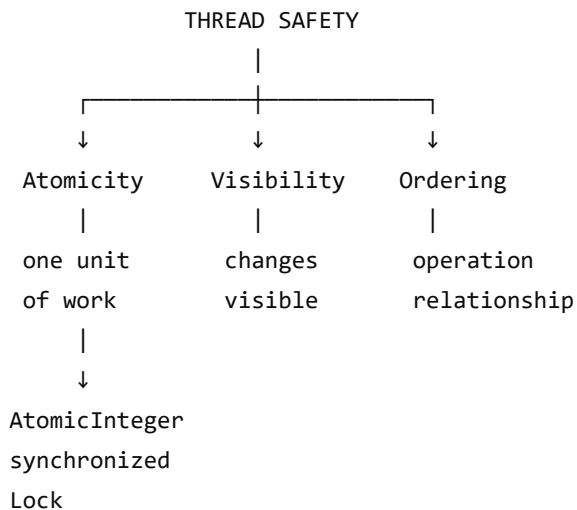
"Which one would you choose for a thread-safe counter?"

A good answer is:

"For a simple increment/read counter, I'd generally use `AtomicInteger` because it provides atomic operations without explicitly managing a lock. If the counter update is part of a larger critical section involving multiple shared variables or operations that must remain consistent together, I'd use `synchronized` or `ReentrantLock`."

🔥 Final Mental Model

Remember these four concepts together:



And:

```
volatile
  ↓
Visibility ✓
Ordering ✓
Atomicity ✗ for compound operations

synchronized
  ↓
Atomicity / Mutual Exclusion ✓
Visibility ✓
Ordering ✓
```

The most common interview trap

```
</> Java

volatile int count;

count++;
```

Not thread-safe.

Because:

```
count++
  =
read
  +
write
```

`volatile` makes the **individual read/write accesses** have the required volatile semantics, but it does not turn the whole read-modify-write sequence into one atomic operation.

For that:

```
</> Java

AtomicInteger count = new AtomicInteger();
count.incrementAndGet();
```

or:

</> **Java**

```
synchronized void increment() {  
    count++;  
}
```

Use the one that matches the complexity of the shared-state operation.



ChatGPT can make mistakes. Check important info.